# Lecture 15 – Continuations (2)

## COSE212: Programming Languages

Jihyeok Park




2026 Fall

# Recall

- We will learn about **continuations** with the following topics:
  - **Continuations** (Lecture 14 & 15)
  - **First-Class Continuations** (Lecture 16)
  - **Compiling with continuations** (Lecture 17)

- A **continuation** represents the **rest of the computation**.
  - Continuation Passing Style (CPS)
  - Interpreter of FAE in CPS

# Recall

PLRG

- We will learn about **continuations** with the following topics:
  - **Continuations** (Lecture 14 & 15)
  - **First-Class Continuations** (Lecture 16)
  - **Compiling with continuations** (Lecture 17)

- A **continuation** represents the **rest of the computation**.
  - Continuation Passing Style (CPS)
  - Interpreter of FAE in CPS

- So far, we have defined **big-step operational (natural) semantics** for our languages.

# Recall

PLRG

- We will learn about **continuations** with the following topics:
  - **Continuations** (Lecture 14 & 15)
  - **First-Class Continuations** (Lecture 16)
  - **Compiling with continuations** (Lecture 17)

- A **continuation** represents the **rest of the computation**.
  - Continuation Passing Style (CPS)
  - Interpreter of FAE in CPS

- So far, we have defined **big-step operational (natural) semantics** for our languages.

- In this lecture, we define **small-step operational semantics** of FAE using **continuations** before moving on to **first-class continuations**.

# Contents


1. Recall
   - Recall: Interpreter of FAE in CPS
   - Recall: Big-Step vs. Small-Step Semantics
2. Small-Step Semantics of FAE
   - Number
   - Addition
   - Multiplication
   - Identifier Lookup
   - Function Definition
   - Function Application
   - Reflexive Transitive Closure
   - Example
3. Semantic Equivalence
4. First-Order Representations of Continuations

# Contents

1. Recall
   - Recall: Interpreter of FAE in CPS
   - Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE
   - Number
   - Addition
   - Multiplication
   - Identifier Lookup
   - Function Definition
   - Function Application
   - Reflexive Transitive Closure
   - Example

3. Semantic Equivalence

4. First-Order Representations of Continuations

# Recall: Interpreter of FAE in CPS

PLRG

In the previous lecture, we represented the **continuation** of each expression in the interpreter of FAE as a **function** and implemented the interpreter in **continuation passing style** (CPS):

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

# Recall: Interpreter of FAE in CPS

In the previous lecture, we represented the **continuation** of each expression in the interpreter of FAE as a **function** and implemented the interpreter in **continuation passing style** (CPS):

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

Then, how can we define the **continuations** for the semantics of FAE?

$$
\begin{array}{lrll}
\text{Values} & \mathbb{V} \ni v & ::= n & (\texttt{NumV}) \\
 & & \mid \langle \lambda x.e, \sigma \rangle & (\texttt{CloV}) \\
\text{Environments} & \sigma & \in \mathbb{X} \xrightarrow{\text{fin}} \mathbb{V} & (\texttt{Env}) \\
\text{Continuations} & & = \text{???} &
\end{array}
$$

# Recall: Big-Step vs. Small-Step Semantics

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \quad \rightarrow \quad 5 * 3 \quad \rightarrow \quad 15$$

# Recall: Big-Step vs. Small-Step Semantics

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\textsc{Mul}\ \dfrac{}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \quad \rightarrow \quad 5 * 3 \quad \rightarrow \quad 15$$

It is possible but non-trivial to represent **continuations** in the **big-step operational (natural) semantics**.

# Recall: Big-Step vs. Small-Step Semantics

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \quad \rightarrow \quad 5 * 3 \quad \rightarrow \quad 15$$

It is possible but non-trivial to represent **continuations** in the **big-step operational (natural) semantics**.

Let's define the **small-step operational semantics** of FAE to represent **continuations**.

# Contents

1. Recall
   - Recall: Interpreter of FAE in CPS
   - Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE
   - Number
   - Addition
   - Multiplication
   - Identifier Lookup
   - Function Definition
   - Function Application
   - Reflexive Transitive Closure
   - Example

3. Semantic Equivalence

4. First-Order Representations of Continuations

# Small-Step Semantics of FAE

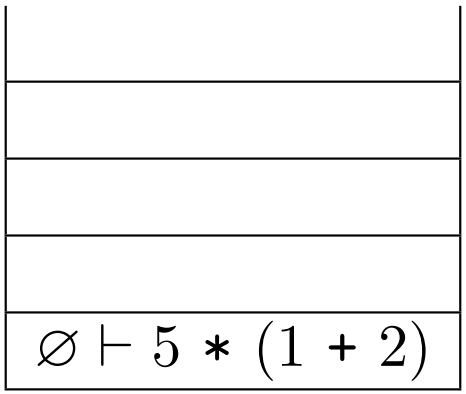
$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:

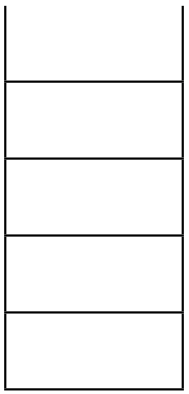
# Small-Step Semantics of FAE

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

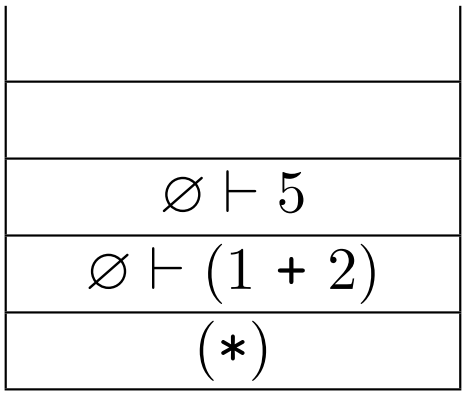


Value Stack

# Small-Step Semantics of FAE

$$\dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \quad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \quad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\textsc{Mul}\ \dfrac{}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

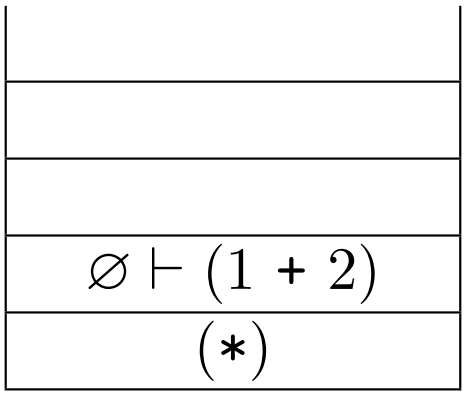


Value Stack

# Small-Step Semantics of FAE

$$\dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \quad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \quad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\textsc{Mul}\ \dfrac{}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

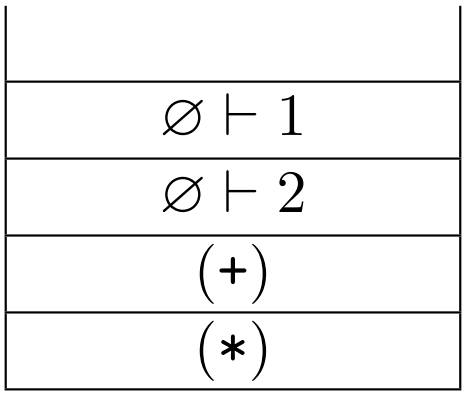



Value Stack

# Small-Step Semantics of FAE

$$\dfrac{\dfrac{}{\varnothing \vdash 5 \Rightarrow 5}\,\textsc{Num} \qquad \dfrac{\dfrac{}{\varnothing \vdash 1 \Rightarrow 1}\,\textsc{Num} \quad \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}\,\textsc{Num}}{\varnothing \vdash 1 + 2 \Rightarrow 3}\,\textsc{Add}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}\,\textsc{Mul}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

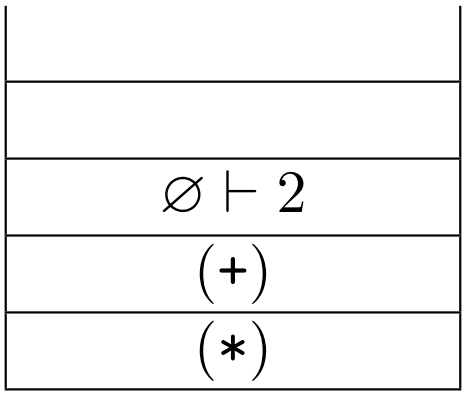



Value Stack

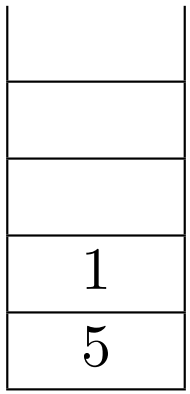
# Small-Step Semantics of FAE

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

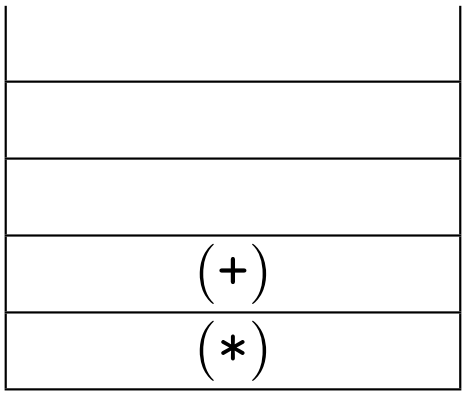



Value Stack

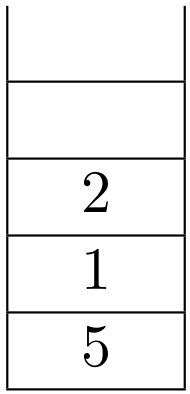
# Small-Step Semantics of FAE

$$\dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\textsc{Mul}\ \dfrac{}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation

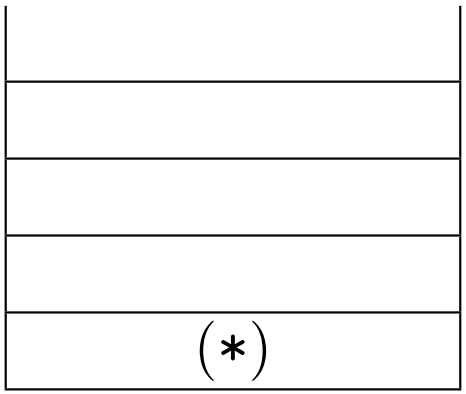



Value Stack

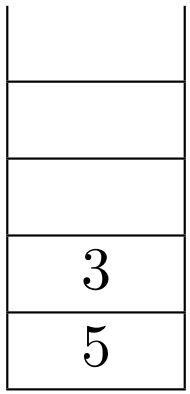
# Small-Step Semantics of FAE

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:




Continuation




Value Stack

# Small-Step Semantics of FAE

$$\textsc{Mul}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 5 \Rightarrow 5} \qquad \textsc{Add}\ \dfrac{\textsc{Num}\ \dfrac{}{\varnothing \vdash 1 \Rightarrow 1} \qquad \textsc{Num}\ \dfrac{}{\varnothing \vdash 2 \Rightarrow 2}}{\varnothing \vdash 1 + 2 \Rightarrow 3}}{\varnothing \vdash 5 * (1 + 2) \Rightarrow 15}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



Continuation




Value Stack

# Small-Step Semantics of FAE

- **Big-step operational (natural) semantics**:

$$\boxed{\sigma \vdash e \Rightarrow v}$$

- **Small-step operational semantics**:

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

where $\rightarrow \subseteq (\mathbb{K} \times \mathbb{S}) \times (\mathbb{K} \times \mathbb{S})$ is a **small-step relation** between **states**.

$$\begin{aligned} \text{Continuations} \quad \mathbb{K} \ni \kappa ::= {} & \square \\ & \mid (\sigma \vdash e) :: \kappa \\ & \mid (\texttt{+}) :: \kappa \\ & \mid (\texttt{*}) :: \kappa \\ & \mid (\texttt{@}) :: \kappa \end{aligned}$$

$$\text{Value Stacks} \quad \mathbb{S} \ni s ::= \blacksquare \mid v :: s$$

# Numbers

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Num(n) => k(NumV(n))
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{Num} \quad \langle (\sigma \vdash n) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid n :: s \rangle$$

# Addition

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Add(l, r) =>
    interpCPS(l, env, {
      lv => interpCPS(r, env, {
        rv => k(numAdd(lv, rv))
      })
    })
```

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle$$

$$\texttt{Add}_1 \quad \langle (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \mid\mid s \rangle$$

$$\texttt{Add}_2 \quad \langle (\texttt{+}) :: \kappa \mid\mid n_2 :: n_1 :: s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid (n_1 + n_2) :: s \rangle$$

# Multiplication

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Mul(l, r) =>
    interpCPS(l, env, {
      lv => interpCPS(r, env, {
        rv => k(numMul(lv, rv))
      })
    })
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{Mul}_1 \quad \langle (\sigma \vdash e_1 \texttt{ * } e_2) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{*}) :: \kappa \mid\mid s \rangle$$

$$\texttt{Mul}_2 \quad \langle (\texttt{*}) :: \kappa \mid\mid n_2 :: n_1 :: s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid (n_1 \times n_2) :: s \rangle$$

# Identifier Lookup

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Id(x) => k(lookupId(x, env))
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{Id} \quad \langle (\sigma \vdash x) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid \sigma(x) :: s \rangle$$

# Function Definition

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Fun(p, b) => k(CloV(p, b, env))
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{Fun} \quad \langle (\sigma \vdash \lambda x.e) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid \langle \lambda x.e, \sigma \rangle :: s \rangle$$

# Function Application

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case App(f, e) => interpCPS(f, env, v => v match
    case CloV(p, b, fenv) =>
      interpCPS(e, env, v => {
        interpCPS(b, fenv + (p -> v), k)
      })
    case v => error(s"not a function: ${v.str}")
  )
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{App}_1 \langle (\sigma \vdash e_1(e_2)) :: \kappa \mid\mid s \rangle \rightarrow \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{@}) :: \kappa \mid\mid s \rangle$$

$$\texttt{App}_2 \langle (\texttt{@}) :: \kappa \mid\mid v_2 :: \langle \lambda x.e, \sigma \rangle :: s \rangle \rightarrow \langle (\sigma[x \mapsto v_2] \vdash e) :: \kappa \mid\mid s \rangle$$

# Reflexive Transitive Closure

To describe the **whole evaluation** of an expression, we need to take **zero or more steps** of $(\rightarrow)$.

# Reflexive Transitive Closure

PLRG

To describe the **whole evaluation** of an expression, we need to take **zero or more steps** of $(\rightarrow)$.

The **reflexive transitive closure** $(\rightarrow^*)$ of $(\rightarrow)$:

$$\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid s \rangle$$

$$\frac{\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa' \mid\mid s' \rangle \qquad \langle \kappa' \mid\mid s' \rangle \rightarrow \langle \kappa'' \mid\mid s'' \rangle}{\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa'' \mid\mid s'' \rangle}$$

# Reflexive Transitive Closure

To describe the **whole evaluation** of an expression, we need to take **zero or more steps** of $(\rightarrow)$.

The **reflexive transitive closure** $(\rightarrow^*)$ of $(\rightarrow)$:

$$\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid s \rangle$$

$$\frac{\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa' \mid\mid s' \rangle \qquad \langle \kappa' \mid\mid s' \rangle \rightarrow \langle \kappa'' \mid\mid s'' \rangle}{\langle \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa'' \mid\mid s'' \rangle}$$

Then, the evaluation of an expression $e$ produces a value $v$ if:

$$\langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \qquad\qquad || \ \blacksquare \qquad\qquad \rangle$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{ll} & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \ \blacksquare \quad \rangle \\ (\underset{\rightarrow}{\texttt{App}_1}) & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \ \blacksquare \quad \rangle \end{array}$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
(\mathtt{App}_1) \rightarrow & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\mathtt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
(\mathtt{Fun}) \rightarrow & \langle\ (\varnothing \vdash 2) :: (\mathtt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\mathtt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\mathtt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\mathtt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\mathtt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\mathtt{Num})]{} & \langle\ (\mathtt{@}) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \ \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ (\texttt{@}) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \ \rangle \\
\xrightarrow[(\texttt{App}_2)]{} & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square \quad || \quad \blacksquare \quad \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ (\texttt{@}) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \ \rangle \\
\xrightarrow[(\texttt{App}_2)]{} & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Add}_1)]{} & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square \quad || \quad \blacksquare \quad \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{rll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square & \|\ \blacksquare\ \rangle \\
(\texttt{App}_1)\ \rightarrow & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (@) :: \square & \|\ \blacksquare\ \rangle \\
(\texttt{Fun})\ \rightarrow & \langle\ (\varnothing \vdash 2) :: (@) :: \square & \|\ \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
(\texttt{Num})\ \rightarrow & \langle\ (@) :: \square & \|\ 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
(\texttt{App}_2)\ \rightarrow & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square & \|\ \blacksquare\ \rangle \\
(\texttt{Add}_1)\ \rightarrow & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square & \|\ \blacksquare\ \rangle \\
(\texttt{Num})\ \rightarrow & \langle\ ([x \mapsto 2] \vdash x) :: (+) :: \square & \|\ 1 :: \blacksquare\ \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{rll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square & ||\ \blacksquare\ \rangle \\
(\texttt{App}_1) \rightarrow & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square & ||\ \blacksquare\ \rangle \\
(\texttt{Fun}) \rightarrow & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square & ||\ \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
(\texttt{Num}) \rightarrow & \langle\ (\texttt{@}) :: \square & ||\ 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
(\texttt{App}_2) \rightarrow & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square & ||\ \blacksquare\ \rangle \\
(\texttt{Add}_1) \rightarrow & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square & ||\ \blacksquare\ \rangle \\
(\texttt{Num}) \rightarrow & \langle\ ([x \mapsto 2] \vdash x) :: (+) :: \square & ||\ 1 :: \blacksquare\ \rangle \\
(\texttt{Id}) \rightarrow & \langle\ (+) :: \square & ||\ 2 :: 1 :: \blacksquare\ \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{rll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square & \|\ \blacksquare\ \rangle \\
\xrightarrow[(\texttt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square & \|\ \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square & \|\ \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ (\texttt{@}) :: \square & \|\ 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare\ \rangle \\
\xrightarrow[(\texttt{App}_2)]{} & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square & \|\ \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Add}_1)]{} & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square & \|\ \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ ([x \mapsto 2] \vdash x) :: (+) :: \square & \|\ 1 :: \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Id})]{} & \langle\ (+) :: \square & \|\ 2 :: 1 :: \blacksquare\ \rangle \\
\xrightarrow[(\texttt{Add}_2)]{} & \langle\ \square & \|\ 3 :: \blacksquare\ \rangle
\end{array}
$$

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ (\texttt{@}) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_2)]{} & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Add}_1)]{} & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ ([x \mapsto 2] \vdash x) :: (+) :: \square \quad || \quad 1 :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Id})]{} & \langle\ (+) :: \square \quad || \quad 2 :: 1 :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Add}_2)]{} & \langle\ \square \quad || \quad 3 :: \blacksquare \quad \rangle
\end{array}
$$

Thus, $\langle (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \ || \ \blacksquare \rangle \rightarrow^* \langle \square \ || \ 3 :: \blacksquare \rangle$.

# Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_1)]{} & \langle\ (\varnothing \vdash \lambda x.(1 + x)) :: (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Fun})]{} & \langle\ (\varnothing \vdash 2) :: (\texttt{@}) :: \square \quad || \quad \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ (\texttt{@}) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \varnothing \rangle :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{App}_2)]{} & \langle\ ([x \mapsto 2] \vdash (1 + x)) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Add}_1)]{} & \langle\ ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square \quad || \quad \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Num})]{} & \langle\ ([x \mapsto 2] \vdash x) :: (+) :: \square \quad || \quad 1 :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Id})]{} & \langle\ (+) :: \square \quad || \quad 2 :: 1 :: \blacksquare \quad \rangle \\
\xrightarrow[(\texttt{Add}_2)]{} & \langle\ \square \quad || \quad 3 :: \blacksquare \quad \rangle
\end{array}
$$

Thus, $\langle (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \ || \ \blacksquare \rangle \rightarrow^* \langle \square \ || \ 3 :: \blacksquare \rangle$.

It means that the evaluation result of $(\lambda x.(1 + x))(2)$ is $3$.

# Contents

1. Recall
    - Recall: Interpreter of FAE in CPS
    - Recall: Big-Step vs. Small-Step Semantics
2. Small-Step Semantics of FAE
    - Number
    - Addition
    - Multiplication
    - Identifier Lookup
    - Function Definition
    - Function Application
    - Reflexive Transitive Closure
    - Example
3. Semantic Equivalence
4. First-Order Representations of Continuations

# Semantic Equivalence

Now, we have **two** different semantics for FAE. For example,

**Big-step operational (natural) semantics**:

$$\textsc{App}\ \frac{\varnothing \vdash \lambda x.(1 + x) \Rightarrow \langle \lambda x.(1 + x), \varnothing \rangle \qquad \varnothing \vdash 2 \Rightarrow 2 \qquad [x \mapsto 2] \vdash 1 + x \Rightarrow 3}{\varnothing \vdash (\lambda x.(1 + x))(2) \Rightarrow 3}$$

**Small-step operational semantics**:

$$\langle (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid 3 :: \blacksquare \rangle$$

Both semantics produce the **same result** $3$ for this expression.

# Semantic Equivalence

Now, we have **two** different semantics for FAE. For example,

**Big-step operational (natural) semantics**:

$$\textsc{App}\ \frac{\varnothing \vdash \lambda x.(1 + x) \Rightarrow \langle \lambda x.(1 + x), \varnothing \rangle \qquad \varnothing \vdash 2 \Rightarrow 2 \qquad [x \mapsto 2] \vdash 1 + x \Rightarrow 3}{\varnothing \vdash (\lambda x.(1 + x))(2) \Rightarrow 3}$$

**Small-step operational semantics**:

$$\langle (\varnothing \vdash (\lambda x.(1 + x))(2)) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid 3 :: \blacksquare \rangle$$

Both semantics produce the **same result** $3$ for this expression.

**Question**: Do they **always** produce the same result for **every** expression?

# Semantic Equivalence

The big-step and small-step semantics are **equivalent** if:

$$\varnothing \vdash e \Rightarrow v \qquad \Longleftrightarrow \qquad \langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$

for all $e \in \mathbb{E}$ and $v \in \mathbb{V}$.

# Semantic Equivalence

PLRG

The big-step and small-step semantics are **equivalent** if:

$$\varnothing \vdash e \Rightarrow v \qquad \Longleftrightarrow \qquad \langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$

for all $e \in \mathbb{E}$ and $v \in \mathbb{V}$.

It is a special case of the following **more general** statement:

$$\sigma \vdash e \Rightarrow v \qquad \Longleftrightarrow \qquad \langle (\sigma \vdash e) :: \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid v :: s \rangle$$

for all $\sigma \in \mathbb{X} \xrightarrow{\text{fin}} \mathbb{V}$, $e \in \mathbb{E}$, $v \in \mathbb{V}$, $\kappa \in \mathbb{K}$, and $s \in \mathbb{S}$.

# Semantic Equivalence – Proof

Let's prove one direction ($\Rightarrow$) of the semantic equivalence:

**Theorem**

If $\sigma \vdash e \Rightarrow v$, then $\langle (\sigma \vdash e) :: \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid v :: s \rangle$ for all $\kappa \in \mathbb{K}$ and $s \in \mathbb{S}$.

# Semantic Equivalence – Proof

Let's prove one direction ($\Rightarrow$) of the semantic equivalence:

**Theorem**

If $\sigma \vdash e \Rightarrow v$, then $\langle (\sigma \vdash e) :: \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid v :: s \rangle$ for all $\kappa \in \mathbb{K}$ and $s \in \mathbb{S}$.

- **Proof**: by **induction** on the derivation of $\sigma \vdash e \Rightarrow v$ (i.e., one case for each big-step rule).

# Semantic Equivalence – Proof

Let's prove one direction ($\Rightarrow$) of the semantic equivalence:

**Theorem**

If $\sigma \vdash e \Rightarrow v$, then $\langle (\sigma \vdash e) :: \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid v :: s \rangle$ for all $\kappa \in \mathbb{K}$ and $s \in \mathbb{S}$.

- **Proof**: by **induction** on the derivation of $\sigma \vdash e \Rightarrow v$ (i.e., one case for each big-step rule).
- **Base cases**: `Num`, `Id`, and `Fun` take exactly **one step**. For example,

$$\langle (\sigma \vdash n) :: \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid n :: s \rangle \quad (\texttt{Num})$$

# Semantic Equivalence – Proof

Let's prove one direction ($\Rightarrow$) of the semantic equivalence:

**Theorem**

If $\sigma \vdash e \Rightarrow v$, then $\langle (\sigma \vdash e) :: \kappa \mid\mid s \rangle \rightarrow^* \langle \kappa \mid\mid v :: s \rangle$ for all $\kappa \in \mathbb{K}$ and $s \in \mathbb{S}$.

- **Proof**: by **induction** on the derivation of $\sigma \vdash e \Rightarrow v$ (i.e., one case for each big-step rule).
- **Base cases**: `Num`, `Id`, and `Fun` take exactly **one step**. For example,

$$\langle (\sigma \vdash n) :: \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid n :: s \rangle \quad (\texttt{Num})$$

- **Inductive cases**: `Add`, `Mul`, and `App` use the **induction hypothesis** (IH) for their premises.

# Semantic Equivalence – Proof (Case `Add`)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

# Semantic Equivalence – Proof (Case Add)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$\langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \qquad\qquad \|\ \ s \qquad\qquad \rangle$$

# Semantic Equivalence – Proof (Case Add)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$\begin{array}{l} \\ (\texttt{Add}_1) \\ \rightarrow \end{array} \begin{array}{lll} \langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa & \| & s\ \rangle \\ \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa & \| & s\ \rangle \end{array}$$

# Semantic Equivalence – Proof (Case Add)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \quad || \quad s \quad \rangle \\
(\texttt{Add}_1)\ \rightarrow & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad s \quad \rangle \\
(\text{IH})\ \rightarrow^* & \langle\ (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad n_1 :: s \quad \rangle
\end{array}$$

# Semantic Equivalence – Proof (Case Add)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \quad || \quad s \quad \rangle \\
(\texttt{Add}_1) \rightarrow & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad s \quad \rangle \\
(\text{IH}) \rightarrow^* & \langle\ (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad n_1 :: s \quad \rangle \\
(\text{IH}) \rightarrow^* & \langle\ (\texttt{+}) :: \kappa \quad || \quad n_2 :: n_1 :: s \quad \rangle
\end{array}$$

# Semantic Equivalence – Proof (Case Add)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$
\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \quad || \quad s \quad \rangle \\
(\texttt{Add}_1)\ \rightarrow & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad s \quad \rangle \\
(\text{IH})\ \rightarrow^* & \langle\ (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad n_1 :: s \quad \rangle \\
(\text{IH})\ \rightarrow^* & \langle\ (\texttt{+}) :: \kappa \quad || \quad n_2 :: n_1 :: s \quad \rangle \\
(\texttt{Add}_2)\ \rightarrow & \langle\ \kappa \quad || \quad (n_1 + n_2) :: s \quad \rangle
\end{array}
$$

# Semantic Equivalence – Proof (Case `Add`)

$$\textsc{Add}\ \frac{\sigma \vdash e_1 \Rightarrow n_1 \qquad \sigma \vdash e_2 \Rightarrow n_2}{\sigma \vdash e_1 \texttt{ + } e_2 \Rightarrow n_1 + n_2}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \quad || \quad s \quad \rangle \\
(\texttt{Add}_1)\ \rightarrow & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad s \quad \rangle \\
(\text{IH})\ \rightarrow^* & \langle\ (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \quad || \quad n_1 :: s \quad \rangle \\
(\text{IH})\ \rightarrow^* & \langle\ (\texttt{+}) :: \kappa \quad || \quad n_2 :: n_1 :: s \quad \rangle \\
(\texttt{Add}_2)\ \rightarrow & \langle\ \kappa \quad || \quad (n_1 + n_2) :: s \quad \rangle
\end{array}$$

- The IH on $e_1$ is used with the continuation $(\sigma \vdash e_2) :: (\texttt{+}) :: \kappa$, not $\kappa$.
- The IH on $e_2$ is used with the value stack $n_1 :: s$, not $s$.
- The `Mul` case is similar.

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad \|\quad s\ \rangle$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\overset{(\texttt{App}_1)}{\rightarrow} \begin{array}{lll} \langle\ (\sigma \vdash e_1(e_2)) :: \kappa & \| & s\ \rangle \\ \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa & \| & s\ \rangle \end{array}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad || \quad s \quad \rangle \\
(\mathtt{App}_1)\ \rightarrow & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad s \quad \rangle \\
(\mathsf{IH})\ \rightarrow^* & \langle\ (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad \langle \lambda x.e, \sigma' \rangle :: s \quad \rangle
\end{array}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathtt{App}_1)} & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad \langle \lambda x.e, \sigma' \rangle :: s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (@) :: \kappa \quad || \quad v_2 :: \langle \lambda x.e, \sigma' \rangle :: s \ \rangle
\end{array}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathtt{App}_1)} & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad \langle \lambda x.e, \sigma' \rangle :: s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (@) :: \kappa \quad || \quad v_2 :: \langle \lambda x.e, \sigma' \rangle :: s \ \rangle \\
\xrightarrow{(\mathtt{App}_2)} & \langle\ (\sigma'[x \mapsto v_2] \vdash e) :: \kappa \quad || \quad s \quad \rangle
\end{array}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad || \quad s \quad\rangle \\
\xrightarrow{(\mathtt{App}_1)} & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad s \quad\rangle \\
\xrightarrow[(\mathsf{IH})]{}{}^{*} & \langle\ (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad \langle \lambda x.e, \sigma' \rangle :: s \quad\rangle \\
\xrightarrow[(\mathsf{IH})]{}{}^{*} & \langle\ (@) :: \kappa \quad || \quad v_2 :: \langle \lambda x.e, \sigma' \rangle :: s \quad\rangle \\
\xrightarrow{(\mathtt{App}_2)} & \langle\ (\sigma'[x \mapsto v_2] \vdash e) :: \kappa \quad || \quad s \quad\rangle \\
\xrightarrow[(\mathsf{IH})]{}{}^{*} & \langle\ \kappa \quad || \quad v :: s \quad\rangle
\end{array}$$

# Semantic Equivalence – Proof (Case App)

$$\textsc{App}\ \frac{\sigma \vdash e_1 \Rightarrow \langle \lambda x.e, \sigma' \rangle \qquad \sigma \vdash e_2 \Rightarrow v_2 \qquad \sigma'[x \mapsto v_2] \vdash e \Rightarrow v}{\sigma \vdash e_1(e_2) \Rightarrow v}$$

$$\begin{array}{ll}
 & \langle\ (\sigma \vdash e_1(e_2)) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathtt{App}_1)} & \langle\ (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (\sigma \vdash e_2) :: (@) :: \kappa \quad || \quad \langle \lambda x.e, \sigma' \rangle :: s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ (@) :: \kappa \quad || \quad v_2 :: \langle \lambda x.e, \sigma' \rangle :: s \quad \rangle \\
\xrightarrow{(\mathtt{App}_2)} & \langle\ (\sigma'[x \mapsto v_2] \vdash e) :: \kappa \quad || \quad s \quad \rangle \\
\xrightarrow{(\mathsf{IH})}{}^{*} & \langle\ \kappa \quad || \quad v :: s \quad \rangle
\end{array}$$

The other direction ($\Leftarrow$) also holds, but its proof is more involved, so we skip it in this course.

# Contents

1. Recall
   - Recall: Interpreter of FAE in CPS
   - Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE
   - Number
   - Addition
   - Multiplication
   - Identifier Lookup
   - Function Definition
   - Function Application
   - Reflexive Transitive Closure
   - Example

3. Semantic Equivalence

4. First-Order Representations of Continuations

# First-Order Representations of Continuations

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

# First-Order Representations of Continuations

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

# First-Order Representations of Continuations

PLRG

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

Let's define the continuations as **data structures** (e.g., algebraic data types) in such languages.

# First-Order Representations of Continuations

PLRG

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

Let's define the continuations as **data structures** (e.g., algebraic data types) in such languages.

We call them the **first-order representations of continuations**.

# First-Order Representations of Continuations

PLRG

```
enum Cont:
  case EmptyK
  case EvalK(env: Env, expr: Expr, k: Cont)
  case AddK(k: Cont)
  case MulK(k: Cont)
  case AppK(k: Cont)

type Stack = List[Value]
```

$$
\begin{array}{lll}
\text{Continuations} & \mathbb{K} \ni \kappa ::= \square & (\texttt{EmptyK}) \\
& \quad \mid (\sigma \vdash e) :: \kappa & (\texttt{EvalK}) \\
& \quad \mid (\texttt{+}) :: \kappa & (\texttt{AddK}) \\
& \quad \mid (\texttt{*}) :: \kappa & (\texttt{MulK}) \\
& \quad \mid (\texttt{@}) :: \kappa & (\texttt{AppK})
\end{array}
$$

$$
\text{Value Stacks} \quad \mathbb{S} \ni s ::= \blacksquare \mid v :: s \quad (\texttt{List[Value]})
$$

# First-Order Representations of Continuations

We define a `step` function that takes a state $\langle \kappa \mid\mid s \rangle$ and **steps** it to another state $\langle \kappa' \mid\mid s' \rangle$ using the small-step relation $\rightarrow$ we defined before:

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa' \mid\mid s' \rangle$$

```
def step(k: Cont, s: Stack): (Cont, Stack) = ???
```

# First-Order Representations of Continuations

PLRG

We define a `step` function that takes a state $\langle \kappa \mid\mid s \rangle$ and **steps** it to another state $\langle \kappa' \mid\mid s' \rangle$ using the small-step relation $\rightarrow$ we defined before:

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa' \mid\mid s' \rangle$$

```
def step(k: Cont, s: Stack): (Cont, Stack) = ???
```

And the `evalK` function **iteratively steps** the state until it reaches the empty continuation $\square$ and returns the single value in the value stack:

```
def evalK(str: String): String =
  import Cont.*
  def aux(k: Cont, s: Stack): Value = step(k, s) match
    case (EmptyK, List(v)) => v
    case (k, s)            => aux(k, s)
  aux(EvalK(Map.empty, Expr(str), EmptyK), List.empty).str
```

$$\langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$

# First-Order Representations of Continuations

PLRG

```
def step(k: Cont, s: Stack): (Cont, Stack) = (k, s) match
  case (EvalK(env, expr, k), s) => expr match
    ...
    case Add(l, r) => (EvalK(env, l, EvalK(env, r, AddK(k))), s)
  ...
  case (AddK(k), r :: l :: s) => (k, numAdd(l, r) :: s)
  ...
```

$$\boxed{\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle}$$

$$\texttt{Add}_1 \quad \langle (\sigma \vdash e_1 \texttt{ + } e_2) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (\texttt{+}) :: \kappa \mid\mid s \rangle$$

$$\texttt{Add}_2 \quad \langle (\texttt{+}) :: \kappa \mid\mid n_2 :: n_1 :: s \rangle \quad \rightarrow \quad \langle \kappa \mid\mid (n_1 + n_2) :: s \rangle$$

Similarly, we can define the `step` function for the other cases.

# Why First-Order Representations?

PLRG

Where is the **rest of the computation** stored in each interpreter?

| **Interpreter** | **Rest of the computation** | **Scala call stack** |
|---|---|---|
| `interp` | implicit (in the Scala call stack) | grows |
| `interpCPS` | Scala functions (`Value => Value`) | grows |
| `evalK` | data ($\kappa$ and $s$) | constant |

# Why First-Order Representations?

PLRG

Where is the **rest of the computation** stored in each interpreter?

| **Interpreter** | **Rest of the computation** | **Scala call stack** |
|---|---|---|
| `interp` | implicit (in the Scala call stack) | grows |
| `interpCPS` | Scala functions (`Value => Value`) | grows |
| `evalK` | data ($\kappa$ and $s$) | constant |

- `aux` in `evalK` is **tail-recursive**, so it is compiled into a **loop**.
  - We can implement it even in C with `structs` and a `while` loop.
  - A deeply nested expression does not overflow the Scala call stack.

# Why First-Order Representations?

PLRG

Where is the **rest of the computation** stored in each interpreter?

| **Interpreter** | **Rest of the computation** | **Scala call stack** |
|---|---|---|
| `interp` | implicit (in the Scala call stack) | grows |
| `interpCPS` | Scala functions (`Value => Value`) | grows |
| `evalK` | data ($\kappa$ and $s$) | constant |

- `aux` in `evalK` is **tail-recursive**, so it is compiled into a **loop**.
  - We can implement it even in C with `structs` and a `while` loop.
  - A deeply nested expression does not overflow the Scala call stack.
- A state $\langle \kappa \mid\mid s \rangle$ is just **data**.
  - We can print, store, and **restore** it at any time.

# Summary

PLRG

1. Recall
   - Recall: Interpreter of FAE in CPS
   - Recall: Big-Step vs. Small-Step Semantics
2. Small-Step Semantics of FAE
   - Number
   - Addition
   - Multiplication
   - Identifier Lookup
   - Function Definition
   - Function Application
   - Reflexive Transitive Closure
   - Example
3. Semantic Equivalence
4. First-Order Representations of Continuations

# Exercise #9

https://github.com/ku-plrg-classroom/docs/tree/main/cose212/fae-cps

- Please see above document on GitHub:
  - Implement `interpCPS` function.
  - Implement `step` function.

- It is just an exercise, and you **don't need to submit** anything.
- However, some exam questions might be related to this exercise, so we **strongly recommend** solving it by yourself.

# Next Lecture

- First-Class Continuations

Jihyeok Park
jihyeok_park@korea.ac.kr
https://plrg.korea.ac.kr